

MAANG CSS Interview Guide

Top 30 Technical Questions & Expert Answers | Architecture, Layout & Performance

Core Layout & Architecture

Q1. Explain the CSS Box Model and how `box-sizing` changes it.

The Box Model wraps every HTML element, consisting of (from inside out): Content, Padding, Border, and Margin. By default (`content-box`), width and height only apply to the content area, meaning padding and borders increase the element's total rendered size. Setting `box-sizing: border-box` forces padding and borders to be included within the defined width and height, preventing unexpected layout expansions.

Q2. How is CSS Specificity calculated?

Specificity is calculated via a four-part weight system (Inline, ID, Class, Element). Inline styles score 1,0,0,0. IDs score 0,1,0,0. Classes, attributes, and pseudo-classes (like `:hover`) score 0,0,1,0. Elements and pseudo-elements (like `::before`) score 0,0,0,1. The `!important` rule overrides regular specificity entirely but should be avoided as it breaks the natural cascade. The universal selector (`*`) has 0 specificity.

Q3. What creates a Stacking Context, and why does z-index sometimes not work?

A Stacking Context is a three-dimensional conceptualization of HTML elements along the Z-axis. `z-index` only works on positioned elements (relative, absolute, fixed, sticky). A new context is formed by elements with absolute/relative positioning and a `z-index` other than `auto` , or by properties like `opacity < 1` , `transform` , `filter` , or `will-change` . Elements within a lower stacking context can never overlap elements in a higher one, regardless of their internal `z-index` .

Q4. What is a Block Formatting Context (BFC)?

A BFC is a mini-layout environment within a document where elements operate independently. It is triggered by `float` (non-none), `position: absolute/fixed` , `display: inline-block/flex/grid` , or `overflow: hidden/auto` . BFCs are crucial for two reasons: they contain internal floats (preventing parent collapse) and they prevent vertical margin collapsing between the elements inside and outside the BFC.

Layout Module	Dimensionality	Primary Use Case	Alignment Model
Flexbox	1-Dimensional (Row or Column)	Distributing space and aligning content linearly.	Content-out (sizes based on items).
CSS Grid	2-Dimensional (Rows and Columns)	Macro-layouts, strict architectural grids.	Container-in (tracks strictly defined).

Rendering & Flow Mechanics

Q5. When does Margin Collapsing occur, and how do you prevent it?

Margin collapsing happens when the vertical margins of adjoining block-level elements combine into a single margin equal to the larger of the two. It occurs between adjacent siblings, or between a parent and its first/last child. It can be prevented by adding padding or a border to the parent, triggering a BFC (e.g., `overflow: hidden`), or using Flexbox/Grid layouts where margins never collapse.

Q6. Explain the difference between pseudo-classes and pseudo-elements.

Pseudo-classes (e.g., `:hover`, `:focus`, `:nth-child`) select elements based on their state or structural position in the DOM. **Pseudo-elements** (e.g., `::before`, `::after`, `::first-line`) create virtual elements that do not exist in the DOM, allowing you to style specific parts of an element or inject cosmetic content via the `content` property.

Q7. How do CSS Variables (Custom Properties) differ from preprocessor variables (Sass)?

CSS Variables (`--var-name`) are evaluated at runtime by the browser, live in the DOM, follow cascading and specificity rules, and can be manipulated via JavaScript (`style.setProperty`). Sass variables (`$var-name`) are evaluated at compile time; the browser only sees the final hardcoded values. CSS Variables are vastly superior for dynamic theming.

Q8. Explain Hardware Acceleration in CSS.

Hardware acceleration offloads certain rendering tasks (like compositing and animations) from the CPU to the GPU, dramatically improving performance. It is triggered by animating specific properties like `transform` and `opacity`. You can hint the browser to prepare an element for GPU acceleration using `will-change: transform`, or historically using the `transform: translateZ(0)` hack.

Q9. Compare techniques for hiding elements on a page.

The method chosen impacts the DOM, the Render Tree, and Accessibility. Interviewers look for deep understanding of how these properties interact with layout calculation and screen readers.

Property	Occupies Layout Space?	Interactive (Clicks)?	Screen Readers?
<code>display: none</code>	No (removed from render tree)	No	Ignored
<code>visibility: hidden</code>	Yes (leaves a blank hole)	No	Ignored
<code>opacity: 0</code>	Yes	Yes (can still trigger events)	Read aloud

Modern Layouts (Grid & Flexbox)

Q10. What does the shorthand `flex: 1` actually mean?

`flex: 1` is a shorthand for `flex-grow: 1`, `flex-shrink: 1`, and `flex-basis: 0%`. This combination tells the flex item to consume all available positive free space evenly, to shrink if necessary to prevent overflow, and to start calculating its size from zero (ignoring its intrinsic content size). This ensures multiple `flex: 1` siblings will be exactly the same width regardless of their content length.

Q11. Explain `auto-fill` vs `auto-fit` in CSS Grid.

Both are used with `repeat()` to create responsive grids without media queries. `auto-fill` will insert as many tracks (columns) as will fit into the container, even if they are empty, leaving blank space at the end. `auto-fit` behaves similarly but goes one step further: it collapses any completely empty tracks to zero width, allowing the populated tracks to stretch and consume the remaining space.

Q12. What is CSS Subgrid?

Subgrid (`grid-template-columns: subgrid`) allows a nested grid child to participate in its parent's grid sizing. Normally, nested grids define their own independent tracks. With subgrid, the nested child aligns its items directly to the lines of the parent grid, making complex card layouts (where headers, bodies, and footers align horizontally across different cards) trivial to build.

Q13. How does `position: sticky` differ from `fixed`?

`position: fixed` removes an element from normal document flow and anchors it relative to the viewport. `position: sticky` is a hybrid: it behaves like `relative` positioning until it crosses a specified threshold (e.g., `top: 0`) during scrolling, at which point it "sticks" like fixed positioning. Crucially, a sticky element remains constrained within its nearest block-level parent container.

Q14. What are CSS Container Queries?

Media queries (`@media`) adapt styles based on the viewport size. Container queries (`@container`) allow an element to adapt its styling based on the size of its nearest containing block. This represents a paradigm shift for component-based architecture, allowing a "Card" component to look different when placed in a narrow sidebar versus a wide main content area, independent of the overall screen size.

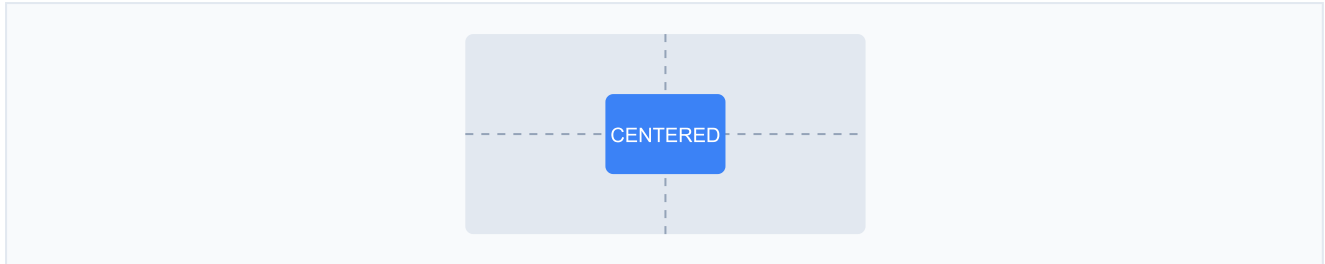
Architecture Focus: The BEM Methodology

Block Element Modifier (BEM) is a naming convention (e.g., `.card__title--large`) designed to make CSS modular, reusable, and predictable in large teams. By enforcing flat specificity (everything is a single class) and strict scoping, BEM prevents the "CSS global namespace" issue, eliminating cascading side-effects when modifying isolated React/Vue components.

Responsive Design & Techniques

Q15. Name three distinct ways to perfectly center an element both vertically and horizontally.

1. **Flexbox:** Apply `display: flex; justify-content: center; align-items: center;` on the parent.
2. **Grid:** Apply `display: grid; place-items: center;` on the parent.
3. **Absolute Positioning:** On the child, set `position: absolute; top: 50%; left: 50%; transform: translate(-50%, -50%);` (assuming relative parent).



Q16. Responsive vs Adaptive Design: What is the difference?

Responsive design uses fluid grids, relative units (percentages, `vw`), and media queries to dynamically adjust content seamlessly to any screen size. Adaptive design uses static layouts defined for specific predetermined screen breakpoints. Responsive is continuous and fluid; adaptive "snaps" to rigid templates as the viewport crosses thresholds.

Q17. Explain the purpose of the `@supports` rule (Feature Queries).

`@supports` allows developers to write CSS conditionals based on browser support for a specific property. Unlike Polyfills or JS detection, it is native. For example, `@supports (display: grid) { ... }` ensures fallback layout code is safely isolated from modern grid code, gracefully degrading the experience on legacy browsers.

Q18. How do CSS Animations differ from CSS Transitions?

Transitions require an explicit state change (like a `:hover` trigger or a JS class toggle) to interpolate between a start and end value. Animations (using `@keyframes`) can execute independently on load, allow for complex multi-step timelines via percentages (0% to 100%), and support properties like looping (`iteration-count`) and direction reversals without user interaction.

Q19. What is the difference between `em` and `rem` units?

Both are relative length units primarily used for typography and spacing. `em` is relative to the font-size of its direct parent element, which can lead to compounding issues in nested structures. `rem` (root em) is relative only to the root `<html>` element's font-size (usually 16px). Using `rem` ensures consistent scaling across the entire application.

Performance & Advanced Concepts

Q20. What is Critical CSS and why does Render-Blocking CSS matter?

The browser pauses DOM construction to download and parse any synchronous stylesheets in the `<head>`, delaying the First Contentful Paint (FCP). Critical CSS is the practice of extracting the absolute minimum CSS required to render "above-the-fold" content and inlining it directly in the HTML. The rest of the CSS is then loaded asynchronously, optimizing performance metrics.

Q21. Explain CSS Containment (`contain`` property).

The `contain` property hints to the browser that an element's internal layout, style, or paint is entirely independent of the rest of the document. For highly dynamic web apps (like dragging items in a virtualized list), setting `contain: strict` restricts layout recalculations to just that sub-tree, preventing expensive document-wide repaints.

Q22. What causes Layout Thrashing and how do you fix it?

Layout thrashing occurs when JavaScript repeatedly alternates between reading DOM geometric properties (like `offsetHeight`) and writing styling changes in a single frame. Reading forces the browser to recalculate layouts synchronously, destroying performance. Fix it by batching all DOM reads together, followed by all DOM writes, typically wrapped inside a `requestAnimationFrame` loop.

Q23. What is the `:has()` pseudo-class?

Historically, CSS only cascaded downwards. `:has()` is the long-awaited "parent selector." It allows you to style a parent element based on its descendants or subsequent siblings. For example, `figure:has(figcaption) { background: grey; }` will style the `figure` only if it contains a `figcaption` inside it. This removes the need for complex JavaScript state tracking in many UI patterns.

Q24. Explain CSS Logical Properties and their use case.

Logical properties map spacing and sizing to the document's writing mode rather than physical screen directions. Instead of `margin-left`, you use `margin-inline-start`. If the page language changes from English (LTR) to Arabic (RTL), `margin-inline-start` automatically applies to the right side without requiring redundant CSS overrides, making internationalization trivial.

Q25. How do CSS Modules work in React/Next.js?

CSS Modules are CSS files where all class names are scoped locally by default. During the build step (via Webpack/Vite), class names like `.button` are hashed into unique identifiers (e.g., `.button_hash123`). This guarantees zero global namespace collisions, allowing developers to write simple, descriptive CSS classes without strict methodologies like BEM.

Edge Cases & Accessibility

Q26. How does the CSS `calc()` function work?

`calc()` allows you to perform mathematical operations natively in CSS. Its superpower is mixing incompatible unit types at runtime. For example, `width: calc(100vw - 50px)` computes a dynamic width that responds to viewport resizes perfectly, something impossible with preprocessors like Sass which can only do math on identical units at compile time.

Q27. Best practices for writing Print Stylesheets?

Print stylesheets (`@media print`) should reset text to dark colors on white backgrounds, remove background images, and hide interactive UI elements like navbars and footers. Crucially, they should utilize page-break controls like `break-inside: avoid` on tables or cards to ensure components aren't arbitrarily chopped in half across physical paper boundaries.

Q28. Why should you avoid `outline: none`?

Removing the focus outline (`outline: none`) without providing an alternative focus state creates a severe accessibility barrier for keyboard-only users, as they cannot visually track where they are on the page. If the default browser ring is ugly, use the `:focus-visible` pseudo-class to style a custom outline only when keyboard navigation is detected, sparing mouse users.

Q29. What is the difference between `line-height: 1.5` and `line-height: 150%`?

While they look identical on the element they are applied to, they cascade differently. A percentage (`150%`) computes the pixel value based on the current element and passes that static pixel value down to children. A unitless number (`1.5`) passes the ratio down, so children recalculate their line-height dynamically based on their own specific font sizes. Always prefer unitless.

Q30. Explain the `backdrop-filter` property.

While `filter` applies effects directly to an element (like blurring an image), `backdrop-filter` applies effects to the area *behind* an element. It is commonly used with semi-transparent backgrounds to create the frosted glass ("glassmorphism") effect on modals and sticky headers without affecting the readability of the text inside the element itself.

Final Interview Tip

In MAANG interviews, CSS questions often mask deeper layout-engine questions. When asked about hiding elements, centering, or layout thrashing, the interviewer isn't just looking for the property name—they are evaluating if you understand **DOM topology, the Render Tree, and hardware accessibility implications**. Always explain the *why* alongside the *how*.